

목차 (Table of Contents)

교수님 아이디어: 2-Stage 모듈 스와핑

아이디어 내용

실제 적용 시 효과 (Alpamayo 기준)

이미 존재하는 메커니즘: device_map="auto"

교수님 아이디어와 device_map="auto"의 차이

단, Alpamayo에서는 device_map="auto"가 동작하지 않음

그래서 우리 연구가 필요한 이유

교수님 면담 자료 #1

작성일: 2026-02-25

교수님 아이디어: 2-Stage 모듈 스와핑

아이디어 내용

- 비전-프리필 과정과 디코딩 과정을 2-stage로 나눠서 VRAM에 로딩/언로딩
- 사용하는 모듈만 VRAM에 올리고, 끝나면 내림
- 이렇게 하면 VRAM 사용량을 절반으로 줄일 수 있지 않을까?

실제 적용 시 효과 (Alpamayo 기준)

단계	모듈	크기 (BF16)
1	Vision Encoder	1.15 GB
2	VLM (LLM)	15.17 GB
3	Diffusion Decoder	4.56 GB
전체 동시 로드		~21 GB
모듈 스와핑 시 Peak	max(1.15, 15.17, 4.56)	~15.17 GB

→ 21GB → 15.17GB로 감소. **약 28% 절감** (절반까지는 아님)

이미 존재하는 메커니즘: `device_map="auto"`

이 아이디어는 HuggingFace Accelerate 라이브러리의 `device_map="auto"` 기능으로 이미 자동화되어 있음.

```
model = AutoModelForCausalLM.from_pretrained("model_name", device_map="auto")
```

device_map="auto"의 동작: 1. 모델 로드 시 VRAM 용량을 확인 2. VRAM에 들어가는 레이어는 GPU에 배치 3. 초과분은 CPU RAM에 배치 (그래도 부족하면 디스크) 4. 추론 시 CPU에 있는 레이어를 실행 차례에 GPU로 이동 → 완료 후 CPU로 반환

교수님 아이디어와 device_map="auto"의 차이

	교수님 아이디어	device_map="auto"
스와핑 단위	모듈 단위 (Vision / VLM / Diffusion)	레이어 단위 (개별 Transformer 레이어)
구현	수동 (명시적 <code>.to(cuda)</code> / <code>.to(cpu)</code>)	자동 (Accelerate 라이브러리)
개념	동일 — “쓸 때만 올리고 내린다”	동일 — “쓸 때만 올리고 내린다”

→ **핵심은 같음:** 필요할 때만 VRAM에 올리고, 끝나면 내리는 방식. device_map이 이를 레이어 단위로 더 세밀하게 자동 수행.

단, Alpamayo에서는 device_map="auto"가 동작하지 않음

실패 원인: Alpamayo의 `fuse_traj_tokens()` 커스텀 연산 - VLM 출력 텐서와 trajectory 토큰 테이블을 `masked_scatter` 로 합치는 과정 - device_map이 텐서를 GPU/CPU에 분산 배치하면 **디바이스 불일치 에러** 발생 - 실제 테스트에서 CUDA assertion error 확인 (연구 방향 1 실험)

그래서 우리 연구가 필요한 이유

1. 기존 device_map="auto"는 Alpamayo 비호환 → **커스텀 오프로딩 구현 필요**
2. 모듈 단위 스와핑만으로는 VLM(15.17GB) > 12GB VRAM → **레이어 단위 스와핑도 필요**
3. 단순 스와핑은 느림 (Unified Memory 기반 273초) → **명시적 전송 + 파이프라이닝 최적화 필요**

→ 모듈 스와핑(교수님 아이디어) + 레이어 스와핑 + pinned memory + 비동기 프리페치를 **조합한 커스텀 구현**이 연구의 핵심